

tunnel-keepalive

tunnel-keepalive.sh

Revision: 2 Last modified: 2026-09-26T13:50:00Z

Overview

`scripts/tunnel-keepalive.sh` is a self-healing supervisor for the macOS podman SSH tunnel. On macOS, podman/vfkit does not forward the `network_mode: host` container ports to the Mac host; `scripts/ensure-macos-tunnel.sh` bridges that gap with an SSH tunnel forwarding ports **7186, 7187, 7189, 9117**. That SSH connection occasionally dies (network drop / VM hiccup), which silently takes the operator's dashboard (<http://localhost:7187>) and the other ports offline until someone notices and re-runs the ensure script.

This supervisor loops in the background, probes the canonical health port **7187** every ~10 seconds, and whenever it is unreachable re-runs `ensure-macos-tunnel.sh` to re-establish all four forwards — auto-healing the stack with no operator intervention. It logs every outage and re-establish to `qa-results/tunnel-keepalive.log`.

It pairs with the hardening added to `ensure-macos-tunnel.sh` itself (`ServerAliveInterval=15, ServerAliveCountMax=3, ExitOnForwardFailure=yes`), which makes a dead SSH connection exit promptly (~45s) instead of hanging half-open — so the supervisor's probe sees the outage quickly.

Prerequisites

- macOS host running the Boba stack via podman (podman machine running, containers up).

- `scripts/ensure-macos-tunnel.sh` present in the same `scripts/` directory (the (re)establish mechanism).
- `bash`, and `curl` (preferred) or `nc` for the reachability probe.

Usage

Run in the background (recommended — survives the shell):

```
nohup bash scripts/tunnel-keepalive.sh >/dev/null 2>&1 & disown
```

To expose the stack on the LAN (matching a `0.0.0.0` tunnel), pass the bind address through — the supervisor forwards it to `ensure-macos-tunnel.sh`:

```
TUNNEL_BIND_ADDR=0.0.0.0 nohup bash scripts/tunnel-keepalive.sh >/dev/null 2>&1 & disown
```

Foreground (Ctrl-C to stop), useful for debugging:

```
bash scripts/tunnel-keepalive.sh
```

Stop the background supervisor:

```
kill "$(cat .git/.tunnel-keepalive.pid)"
```

Environment variables

Variable	Default	Meaning
TUNNEL_BIND_ADDR	127.0.0.1	Bind address forwarded to <code>ensure-macos-tunnel.sh</code> .
KEEPALIVE_INTERVAL	10	Seconds between health checks.
KEEPALIVE_HEALTH_PORT	7187	Port used for the reachability probe.
KEEPALIVE_LOG	<project>/qa-results/tunnel-keepalive.log	Logfile path.

Edge cases

- **Two supervisors:** a pidfile (`.git/.tunnel-keepalive.pid`, falling back to the project root if `.git` is absent) makes a second instance

refuse to start (“another supervisor is already running”). Stale pidfiles (whose pid is no longer alive) are reclaimed automatically.

- **Probe always against loopback:** the health check probes `127.0.0.1:7187` regardless of `TUNNEL_BIND_ADDR`; the loopback address is reachable for both the `127.0.0.1` and `0.0.0.0` binds, so a LAN-exposed tunnel is still correctly monitored.
- **No probe tool:** if neither `curl` nor `nc` is present, `is_up` fails closed (treats the tunnel as down) rather than falsely reporting it healthy.
- **Re-establish failure:** if `ensure-macos-tunnel.sh` exits non-zero or 7187 is still down afterwards, it is logged and retried on the next cycle — the supervisor never gives up silently.
- **Host-safety:** the loop sleeps between checks (no busy-loop); observed CPU is 0.0%.

Internal behaviour

1. Resolve `SCRIPT_DIR / PROJECT_ROOT`; locate `ensure-macos-tunnel.sh`.
2. Acquire the single-owner pidfile (or exit if a live supervisor holds it).
3. Install an `EXIT/INT/TERM` trap that removes the pidfile (only if ours) and logs shutdown.
4. Loop: probe `127.0.0.1:<health-port>`; on failure, run `TUNNEL_BIND_ADDR=<bind> bash ensure-macos-tunnel.sh`, then re-probe and log the result; sleep `<interval>`; repeat.

Related scripts

- `scripts/ensure-macos-tunnel.sh` — establishes / re-establishes the SSH tunnel (the mechanism this supervisor invokes). Now hardened with SSH `keepalive + ExitOnForwardFailure`.

Last verified date

2026-06-14 — verified end-to-end on macOS: started the supervisor, killed the live tunnel ssh process (`curl 7187 → 000`), and observed auto-reconnect within ~4s (`curl 7187 → 200`), with the re-establish recorded in `qa-results/tunnel-keepalive.log`.